

Python Cache Cheatsheet

Master cachetools for Faster & Smarter Python Apps

Why Cache?
Reduce expensive computations, API calls & DB hits.

Faster Apps
Lower latency, better performance & user experience.

Less Load
Reduce server load & costs.

Easy with cachetools
Simple, powerful & flexible.

1 LRU Cache (Least Recently Used)

```python
from cachetools import LRUCache, cached

cache = LRUCache(maxsize=3)

@cached(cache)
def compute(x):
    print(f"Computing {x}...")
    return x * x

# Ejecución de ejemplo
print(compute(1))   # Computing 1... 1
print(compute(2))   # Computing 2... 4
print(compute(3))   # Computing 3... 9
print(compute(1))   # 1 (cached)
print(compute(4))   # Computing 4... 16
print(compute(2))   # 4 (cached)
print(compute(3))   # 9 (cached)
print(compute(1))   # 1 (cached)
```

Descarta el elemento menos recientemente usado cuando el caché está lleno.

Flujo LRU (maxsize=3)

put(1)	[1]			
put(2)	[1]	[2]		
put(3)	[1]	[2]	[3]	
get(1)	[2]	[3]	[1]	(1 es más reciente)
put(4)	[3]	[1]	[4]	(2 es removido)
get(2)	[3]	[1]	[4]	(miss)
get(1)	[3]	[4]	[1]	

2 TTL Cache (Time To Live)

```python
from cachetools import cached, TTLCache
import time

mi_cache = TTLCache(maxsize=50, ttl=1)  # 1 seg

@cached(mi_cache)
def consulta_pesada(id_producto: int):
    print(f"--- Consultando DB para {id_producto} ---")
    time.sleep(2)  # Simula lentitud
    return f"Datos del producto {id_producto}"

print(consulta_pesada(1123456))
print(consulta_pesada(1123456))  # Cached
print(consulta_pesada(1123456))  # Cached
time.sleep(3)  # Espera a que expire (ttl=1s)
print(consulta_pesada(1123456))  # Vuelve a consultar
```

Los items expiran automáticamente después del tiempo (ttl) definido. Ideal para datos temporales.

3 LFU Cache (Least Frequently Used)

```python
from cachetools import LFUCache, cached

lfu_cache = LFUCache(maxsize=3)

@cached(lfu_cache)
def get_data_lfu(key):
    print(f"Fetching {key} (LFU)...")
    return f"Data for {key}"
```

Frecuencia de uso

		A 1	B 1	C 1
get(A)	→	A 2	B 1	C 1
get(B)	→	A 2	B 2	C 1
put(D)	→	A 2	B 2	D 1
		(C es removido: menor frecuencia)		
get(A)	→	A 3	B 2	D 1

Descarta el menos usado en términos de frecuencia. En empate, el menos reciente.

4 Custom Cache Key Generation

Crea claves personalizadas para manejar argumentos complejos o usar solo un subconjunto de ellos.

```python
from cachetools import LRUCache, cached
import json

custom_key_cache = LRUCache(maxsize=2)

def custom_key_generator(list_arg, dict_arg, simple_arg):
    sorted_dict_str = json.dumps(dict_arg, sort_keys=True)
    list_tuple = tuple(list_arg)
    return (list_tuple, sorted_dict_str, simple_arg)

@cached(custom_key_cache, key=custom_key_generator)
def process_complex_data(data_list, settings_dict, operation_type):
    print(f"Processing {operation_type} with {data_list} & {settings_dict}")
    result = sum(data_list) * len(settings_dict) * (1 if operation_type == "sum" else -1)
    return f"Result for {operation_type}: {result}"
```

Ejemplo de clave generada

data_list	[1, 2]
settings_dict	{"mode": "fast"}
operation_type	"sum"

↓

Clave interna de caché

```
([1, 2],
 {"mode": "fast"},
 "sum")
```

Útil cuando tienes listas, diccionarios o quieres controlar qué argumentos forman parte de la clave.

5 Manual Cache Invalidation

Invalida entries especificas o limpia todo el caché cuando sea necesario.

```python
from cachetools import LFUCache, cached
manual_invalidation_cache = LRUCache(maxsize=5)

@cached(manual_invalidation_cache)
def get_user_profile(user_id):
    print(f"Fetching user {user_id} from DB...")
    return {'id': user_id, 'name': f'User {user_id}', 'status': 'active'}
```

1. Invalida un entry específico

```python
del manual_invalidation_cache[(1.,]
# (Las claves son tuplas de argumentos)
```

Antes	Después de eliminar (1)
(1) (2) (3)	(2) (3)
→ size: 3 —	size: 2

2. Limpia todo el caché

```python
manual_invalidation_cache.clear()
```

Antes	Después de clear()
(2) (3)	[ ]
size: 2	size: 0

Ideal después de actualizaciones de datos, cambios de configuración o eventos del sistema.

RR Cache (Random Replacement)

Reemplaza un item aleatorio cuando el caché está lleno.

```python
from cachetools import RRCache, cached

rr_cache = RRCache(maxsize=3)

@cached(rr_cache)
def get_data_rr(key):
    print(f"Fetching {key} (RR)...")
    return f"Data for {key}"
```



Flujo (maxsize=3)

put(X)	[X]			
put(Y)	[X]	[Y]		
put(Z)	[X]	[Y]	[Z]	(lleno)
put(P)	[?]	[Y]	[Z]	→ (uno se elimina al azar)
put(Q)	[?]	[?]	[?]	→ (otro se elimina al azar)

Buena opción cuando no necesitas un orden específico de descarte.

7 FIFO Cache (First-In First-Out)

Descarta el item más antiguo (el primero en entrar).

```python
from cachetools import FIFOCache,cached

fifo_cache = FIFOCache(maxsize=3)

@cached(fifo_cache)
def get_data_fifo(key):
    print(f"Fetching {key} (FIFO)...")
    return f"Data for {key}"
```

Flujo FIFO (maxsize=3)

put(10)	[10]			
put(20)	[10]	[20]		
put(30)	[10]	[20]	[30]	(lleno)
put(10)	[10]	[20]	[30]	(hit)
put(40)	[20]	[30]	[40]	(10 es removido)
put(20)	[20]	[30]	[40]	(hit)
put(50)	[30]	[40]	[50]	(20 es removido)

Simple y predecible: primero en entrar, primero en salir.

Comparison Overview

Estrategia	Criterio de descarte	Ideal para
LRU	Menos recientemente usado	Datos con patrones de acceso reciente
TTL	Expiración por tiempo	Datos temporales, APIs, sesiones
LFU	Menos usado (frecuencia)	Datos populares y reutilizados often
RR	Aleatorio	Casos simples o con baja precisión
FIFO	Más antiguo (primero en entrar)	Flujos, colas, procesos batch

Elipe la estrategia según el patrón de acceso y la naturaleza de tus datos.

Best Practices

Define el tamaño adecuado
Usa maxsize acorde a la memoria disponible.

Monitorea & Ajusta
Revisa hit rate, tamaño y ajusta según resultados.

Invalida con cuidado
No olvides limpiar o invalidar cuando los datos cambien.

Evita sobre-cachear
No todo necesita caché. Evalúa el costo vs beneficio.

Usa claves claras
Claves simples y consistentes evitan bugs y duplicados.

Menos espera. Más rendimiento. Código más inteligente.

Happy Caching!